

CIS6003_20283220_SE19_37_WRIT1.docx

by Ayyash Mohamed Reeza

Submission date: 05-Sep-2026 01:12PM (UTC+0100)

Submission ID: 288331768

File name: 125799_Ayyash_Mohamed_Reeza_CIS6003_20283220_SE19_37_WRIT1_3150916_1001933755.docx (801.95K)

Word count: 5757

Character count: 38552

1 Introduction

1.1 Context and Background

Sunrise Dental Clinic is a busy private dental clinic which is based in Colombo. It handles large number of patients and appointments every week. Currently, all the patient records, appointment details as well as the treatment records are done through paper files and notebooks.

This paper-based system causes problems.

The existing manual system is slow, inefficient and prone to human error such as double booking, misplaced patient records, and billing mistakes. This can cause long waiting times and poor service.

1.2 Solution

In order to solve these problems, the clinic requires a computerized Appointment and Patient Management System. The proposed system will be developed using Java which will be easy for authorized clinic staff to use. This will allow the clinic staff to log in securely, register and search appointments, calculate bills, print receipts, read help instructions and exit the system safely. It will also use a database, web services and suitable design patterns. This new computerized system will keep information in one place, reduce appointment conflicts, make billing more accurate and help staff find records quickly.

1.3 Objectives

The main aim of this proposal is to develop an effective computerized Appointment and Patient Management System for Sunrise Dental Clinic. It will provide secure login, appointment registration and searching, patient and dentist details, appointment validation, bill calculation, receipt printing and help instructions. It will also use a database, web services, design patterns, OOP, automated testing and Git/GitHub.

1.4 Scope and Limitations

This computerized system will include staff login, patient management, appointment management, dentist and treatment details, billing receipts, and basic reports. However, this will not include online booking, online payments, laboratory management, medical-image

storage, and advanced analytics. These advanced features can be considered in future improvements.

2 System Design and UML Diagrams

2.1 Object-Oriented Design Methodology

Sunrise Dental Clinic Management System uses a solid object-oriented design, sticking to the basic OOP stuff like keeping things private, inheriting traits, being able to change behavior, and simplifying complex stuff. this way of doing things makes sure things are easy to change, keep up with, and grow, plus it keeps different parts of the project neatly apart using proven design tricks (Bouzidi et al., 2020) (Imane Essebaa and Salima Chantit, 2018) (Jick Yeung, 2017) (Creately, 2022) (Visual Paradigm, 2019).

the system's design sticks to the usual UML stuff, using diagrams to show what the system looks like and how it works, both when it's still and when it's in action. the design methodology uses formal UML standards to keep the system docs clear and consistent (Muhamad, Abdulmonim and Alathari, 2019) (Singh, Sharma and Saxena, 2016) (tutorialspoint.com, 2019).

```
<<include>> s.Validate_Credentials because username and password verification event takes place whenever a staff member attempts to log in
```

Register New Appointment: This is one of the core use cases. The system records the appointment number, patient name, address, contact number, dentist, treatment type, date and time required by the assessment. Registration <<include>>s appointment validation and availability checking. The system should reject missing or invalid information, duplicated appointment numbers and conflicting dentist appointments.

Checking dentist availability is a particularly important design decision because the existing manual system is stated to suffer from double bookings.

Search / Display Appointment: Staff enter the unique appointment number and the system retrieves the corresponding appointment and patient information. If the appointment does not exist, an appropriate message should be shown instead of displaying incorrect or empty information.

Calculate Bill: The system retrieves the appropriate treatment charge and consultation fee. The basic business rule is:

$$\text{Total Bill} = \text{Treatment Fee} + \text{Consultation Fee}$$

This reflects the billing requirement stated in the current brief.

Print Bill / Receipt: Printing is modelled as an <<extend>> relationship from Calculate Bill because the calculation can be completed first and the employee can then choose to produce a printed receipt.

Generate Reports: This additional use case supports Task B, which asks for reports that add value to the system. Possible reports include daily appointments, dentist schedules, treatment summaries and revenue summaries.

Help and Exit: The Help Section gives new employees instructions on operating the application, while Exit safely closes the system. Both functions are expressly required by the scenario.

2.2.4 Design Decision Justification

The <<include>> relationship is used for behaviour that is required as part of another function. For example, login cannot be successfully completed without validating credentials. Similarly, an appointment should not be registered without validation.

The <<extend>> relationship is used for receipt printing because this is additional behaviour that follows a completed bill calculation.

The database has deliberately not been modelled as an actor because it is an internal part of the application architecture rather than an external user of the system (Jick Yeung, 2017) (Anon, n.d.).

2.3 Class Diagram

2.3.1 Visual Class Diagram

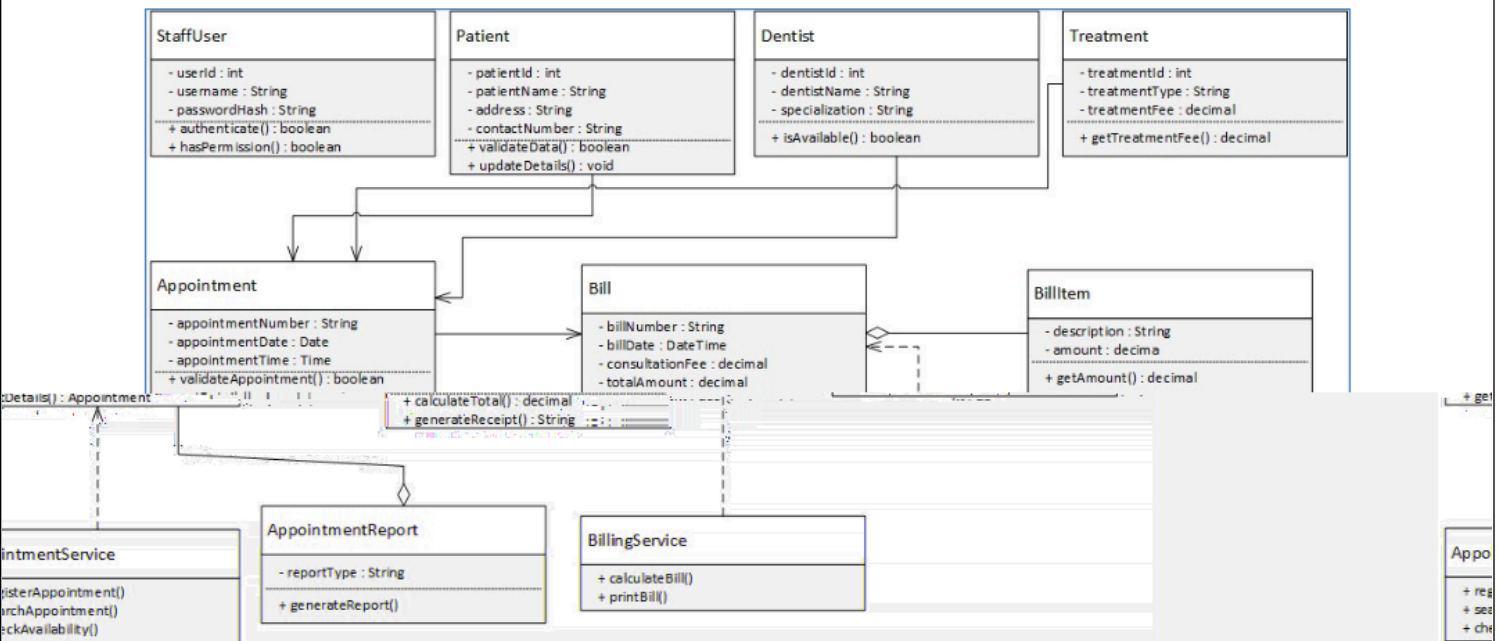


Figure 2-Sunrise Dental Clinic Class Diagram

Figure

The design uses - to identify private attributes and + to identify public operations. This makes encapsulation visible within the UML model. The diagram also contains multiplicities, navigability, composition and aggregation because these features are specifically identified in the excellent grade criteria.

2.3.2 Class Design Analysis

2.3.2.1 Patient

Patient represents a person receiving treatment from Sunrise Dental Clinic. It stores the patient ID, name, address and contact number. Patient data is separated from appointment information because one patient may return to the clinic for several different visits.

2.3.2.2 Dentist

Dentist represents the dentist assigned to an appointment. The `isAvailable()` operation supports scheduling validation and helps prevent a dentist from being allocated to two appointments at the same time.

2.3.2.3 Treatment

Treatment represents the dental treatment selected for an appointment. It contains the treatment type and treatment fee required for billing.

2.3.2.4 Appointment

Appointment is the central entity of the system. It connects a Patient, Dentist and Treatment and records the appointment number, date and time. The appointment number is treated as unique because the brief requires staff to search using this value.

2.3.2.5 StaffUser

StaffUser contains the account information required for authentication. Passwords are represented as passwordHash rather than plain text in the design because storing readable passwords would create an unnecessary security risk.

2.3.2.6 Bill and BillItem

Bill stores the bill number, bill date, consultation fee and calculated total. BillItem represents an individual charge shown on the bill.

The relationship between these two classes is modelled as composition:

Bill 1 ◆— 1.. BillItem*

A BillItem belongs to its Bill and would not normally exist independently of it.

2.3.2.7 AppointmentService and BillingService

The service classes separate business logic from the UI. AppointmentService handles appointment registration, searching and availability checking, while BillingService performs bill calculation and printing.

2.3.2.8 AppointmentReport

AppointmentReport represents a report created from appointment records. Its relationship with appointments is shown as aggregation because deleting a report must not delete the underlying appointment records.

2.3.3 Relationship Analysis

The main multiplicities are:

Relationship	Multiplicity	Explanation
Patient → Appointment	1 : 0..*	One patient may have several appointments
Dentist → Appointment	1 : 0..*	One dentist may handle several appointments
Treatment → Appointment	1 : 0..*	One treatment type may be used in several appointments
Appointment → Bill	1 : 0..1	An appointment may generate one bill
Bill → BillItem	1 : 1..*	A bill contains one or more billing entries
AppointmentReport → Appointment	1 : 0..*	A report may reference several appointments

Navigability arrows indicate which objects require direct access to other objects. Composition is used only where there is strong lifecycle ownership, while aggregation represents a weaker whole-part relationship.

2.5 Sequence Diagrams

Three sequence diagrams have been selected because the marking criteria refer to implementing approximately three identified use cases as sequence diagrams.

2.5.1 Staff Login Sequence

Visual Sequence Diagram:

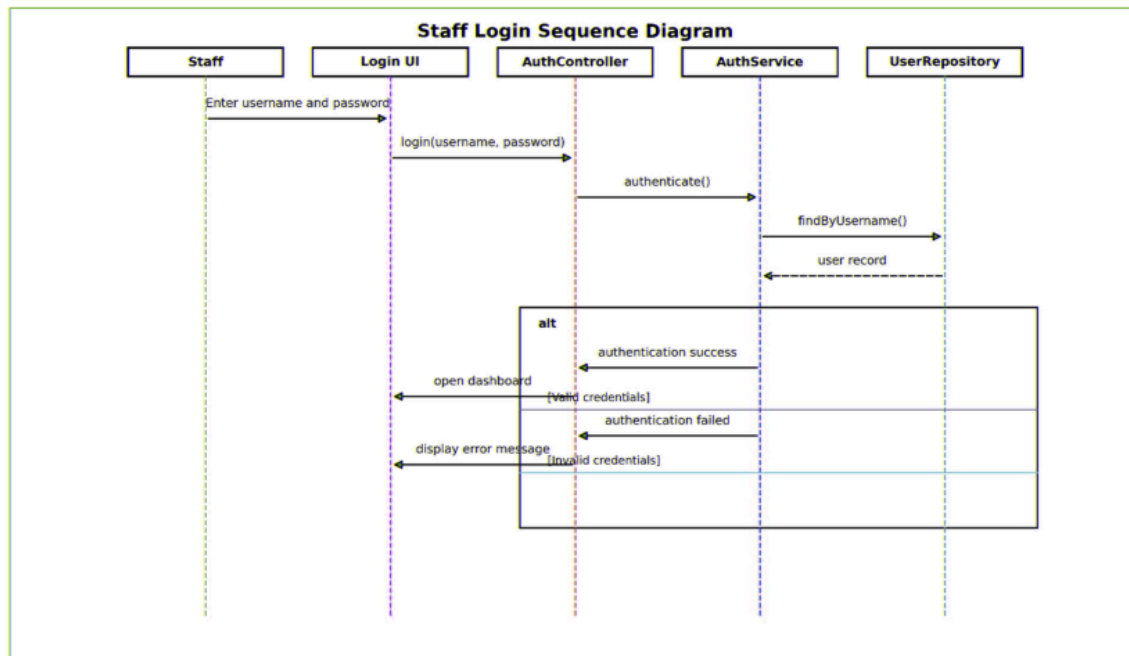


Figure 3-Sequence_Login

Figure 3 shows the authentication process.

The employee enters a username and password through LoginUI. The request is passed to AuthController, which sends it to AuthService. The service retrieves the corresponding user through UserRepository.

An alt fragment represents the two possible results. Valid credentials allow the dashboard to open, whereas invalid credentials cause an error message to be displayed.

This arrangement also demonstrates separation of responsibilities because the graphical user interface does not directly access the database (Sparxsystems.com, 2025) (Sequence Diagrams, n.d.) (sparxsystems.com, n.d.).

2.5.2 Register New Appointment Sequence

Visual Sequence Diagram:

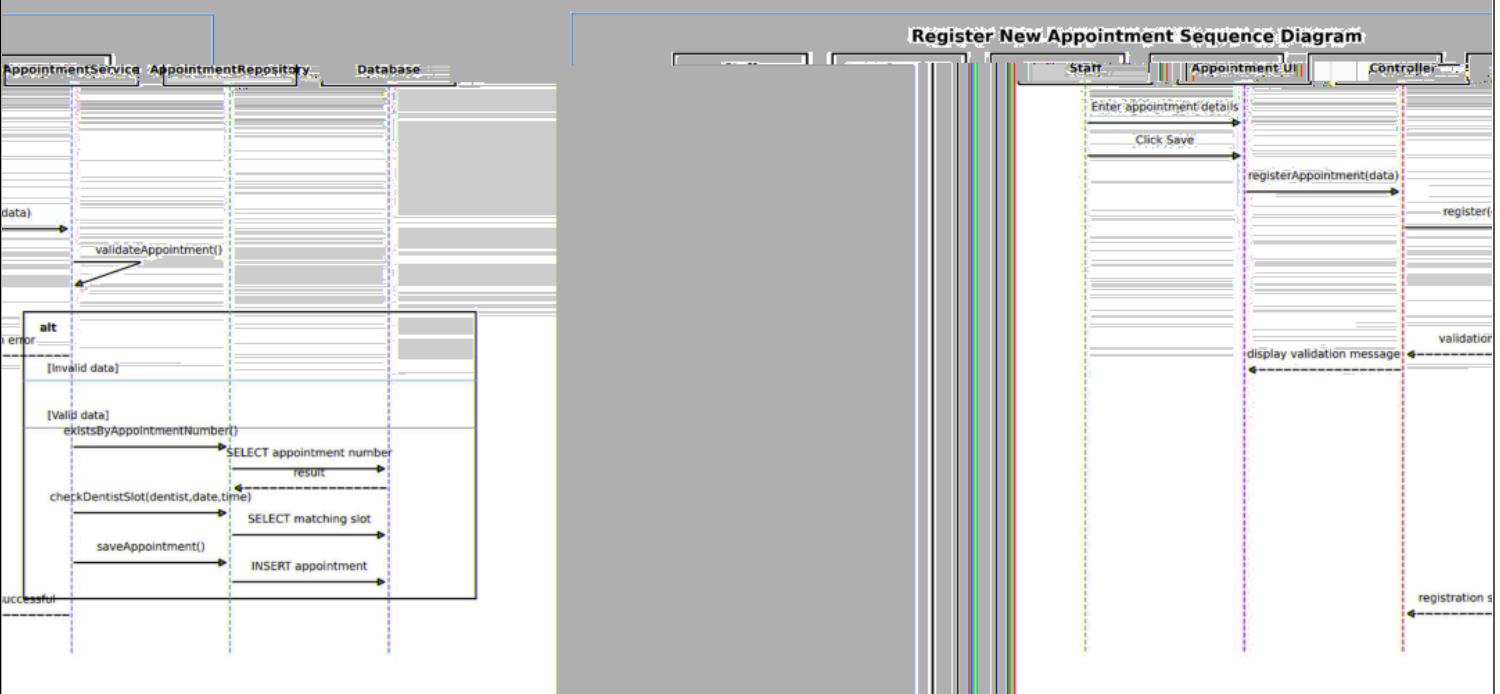


Figure 4-Sequence Register Appointment

Figure 4 represents the registration process.

ation and submits the form. Appointment
The system then checks whether the
the selected dentist is free at the requested
nce Diagrams, n.d.).

ent saved to the database.

problems of inaccurate records and double
ing paper process into electronic form.

The staff member enters the appointment inform
Service first validates the supplied information.
appointment number already exists and whether
date and time (Sparxsystems.com, 2025) (Seque

Only after these checks succeed is the appointm

This design directly responds to the case-study p
booking rather than simply transferring the exist

2.5.3 Search, Calculate and Print Bill Sequence

Visual Sequence Diagram:

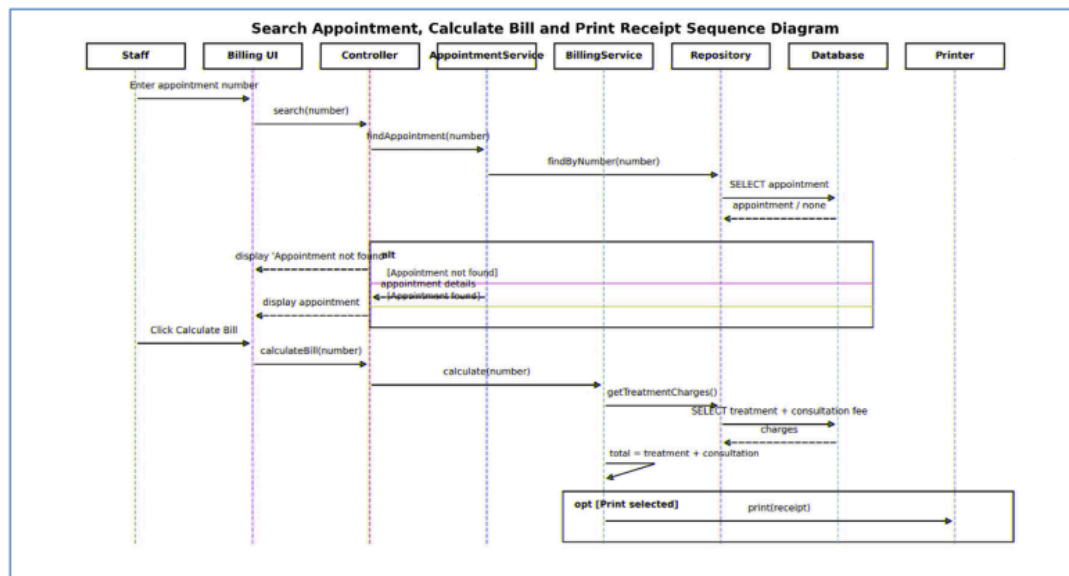


Figure 5-Sequence_Search_Billing_Print

Figure 5 combines the closely connected appointment-search and billing processes.

First, the appointment is retrieved using its appointment number. An alt fragment handles whether the record exists.

Where the appointment is found, the billing service retrieves the appropriate charges and performs:

Total Amount = Treatment Fee + Consultation Fee

The calculated bill is returned to the interface. Printing is represented as an optional operation because the bill can first be calculated and displayed before being sent to the printer.

2.6 ¹ Design Decision Justification

2.6.1 UML Standards Compliance

The UML diagrams adhere to OMG UML 2.5 specifications, utilizing standard notation for actors, use cases, classes, associations, and sequence interactions. The use of stereotypes (<<include>>, <<extend>>) follows established UML conventions for use case relationships (Jick Yeung, 2017) (Anon, n.d.) (tutorialspoint.com, 2019).

2.6.2 Object-Oriented Principles

The design demonstrates clear application of OOP principles:

- **Encapsulation:** Private attributes with public accessor methods ensure data hiding and controlled access (Creately, 2022) (Visual Paradigm, 2019)
- **Inheritance:** Potential for extending Admin class for different user roles
- **Polymorphism:** Method overriding in different entity classes for specialized behavior
- **Abstraction:** Clear separation between interface and implementation through well-defined method signatures

2.6.3 Design Pattern Integration

The design prepares the project for the patterns that will be documented in Task B. The proposed structure is compatible with an MVC/layered approach, where the user interface is separated from business logic and data access.

The service layer centralizes appointment and billing rules, while Repository/DAO classes can isolate database operations (Visual Paradigm, 2019) (www.uml-diagrams.org, n.d.).

¹ 2.6.4 Database Integration

The sequence diagrams demonstrate proper database interaction patterns with connection management, prepared statements for security, and transaction handling for data integrity. The use of parameterized queries prevents SQL injection vulnerabilities while maintaining performance through connection pooling.

This comprehensive UML design provides a solid foundation for system implementation, ensuring scalability, maintainability, and adherence to software engineering best practices (Bouzidi et al., 2020) (Singh, Sharma and Saxena, 2016).

3 Implementation

The ² Sunrise Dental Clinic Appointment and Patient Management System was developed as a Java-based application to improve the management of patients, appointments, treatments and prescriptions within the clinic. The system provides a set of separate graphical interfaces so that authorized staff can perform daily activities through a clear and organized workflow.

The current implementation contains a Login interface, Dashboard, Patient Management interface, Appointment Management interface, Treatment Management interface and Prescription Management interface. Navigation options are provided through a consistent side menu, allowing users to move between the main system modules without returning to the login page.

3.1 System Architecture Overview

The system is divided into several functional modules according to the main activities carried out by the dental clinic.

The main modules implemented through the user interface are:

- User Authentication
- Dashboard
- Patient Management
- Appointment Management
- Treatment Management
- Prescription Management
- Logout and Navigation

This separation improves usability because each interface focuses on one main area of clinic operations instead of placing all functions on a single screen.

The intended overall architecture can be represented as:

- User Interface → Application Logic → Data Access → Database

For the final distributed implementation, a web-service layer should also be placed between the client application and the server-side business logic:

- Client Interface → Web Service/API → Business Logic → Data Access → Database

3.3 User Interface Development

User Interface Development

The graphical user interface has been designed using a consistent theme throughout the application. The Sunrise Dental Clinic logo appears on the left-hand navigation area and the same blue-and-white visual style is maintained across the main screens.

The navigation panel provides access to the relevant modules such as:

- Patients
- Prescriptions
- Appointment
- Treatment
- Dashboard
- Logout

The use of a consistent navigation panel helps staff move between sections without having to repeatedly close and reopen different parts of the system.

3.3.1 Login Interface

The Login interface is the entry point to the application and is designed to restrict access only to authorized staff.

The interface contains:

1. User Name text field
2. Password field
3. Login button
4. Reset option
5. Close button

The password field masks the entered password characters so that the password is not openly displayed on the screen.

When a user enters their credentials and selects Login, First the system is expected to verify the information. Then providing access to the main application.

The Reset option allows the entered values to be cleared if the user wishes to re-enter their username and password.

The login screen also follows a simple design. Only the information required for authentication is displayed, which reduces unnecessary complexity for users.

3.3.2 Dashboard Interface



Figure 7-Main Dashboard

The Dashboard provides the main navigation point after successful authentication.

The left-hand navigation panel gives access to:

- Patients
- Prescriptions
- Appointment
- Treatment
- Logout

The dashboard also visually presents three major areas:

- **Treatments**
- **Patients**
- **Prescriptions**

Each section contains an icon and space for displaying the corresponding number of records.

The purpose of the dashboard is to provide users with a quick summary of the clinic's main operational information and direct access to the principal modules.

For example, once connected to the final database, the dashboard can display information such as:

- Patients: 25
- Treatments: 8
- Prescriptions: 18

This would allow clinic staff to obtain a basic overview without manually opening each section.

3.3.3 Patient Management Interface

The screenshot shows a web application for 'SUNRISE DENTAL'. On the left is a purple sidebar with a logo at the top and navigation links: 'Appointment', 'Treatment', 'Prescription', 'Dashboard', and 'Logout'. The main area is titled 'Patients' and has a red close button in the top right. It contains a form for adding a new patient with fields for 'Name', 'Address', 'DOB', 'Allergies', 'Phone', and 'Gender'. Below the form is a 'Patients List' table with columns: 'ID', 'Patient Name', 'Phone', 'Address', 'DOB', 'Gender', and 'Allergies'. The table is currently empty.

8

Figure 8-Patient Management Interface

The Patient Management interface is used to collect and display information relating to patients registered at Sunrise Dental Clinic.

The interface currently contains the following fields:

- **Name**
- **Phone**
- **Address**
- **Date of Birth**
- **Gender**
- **Allergies**

This provides more information than the minimum patient details required in the scenario and allows the clinic to maintain useful information about each patient.

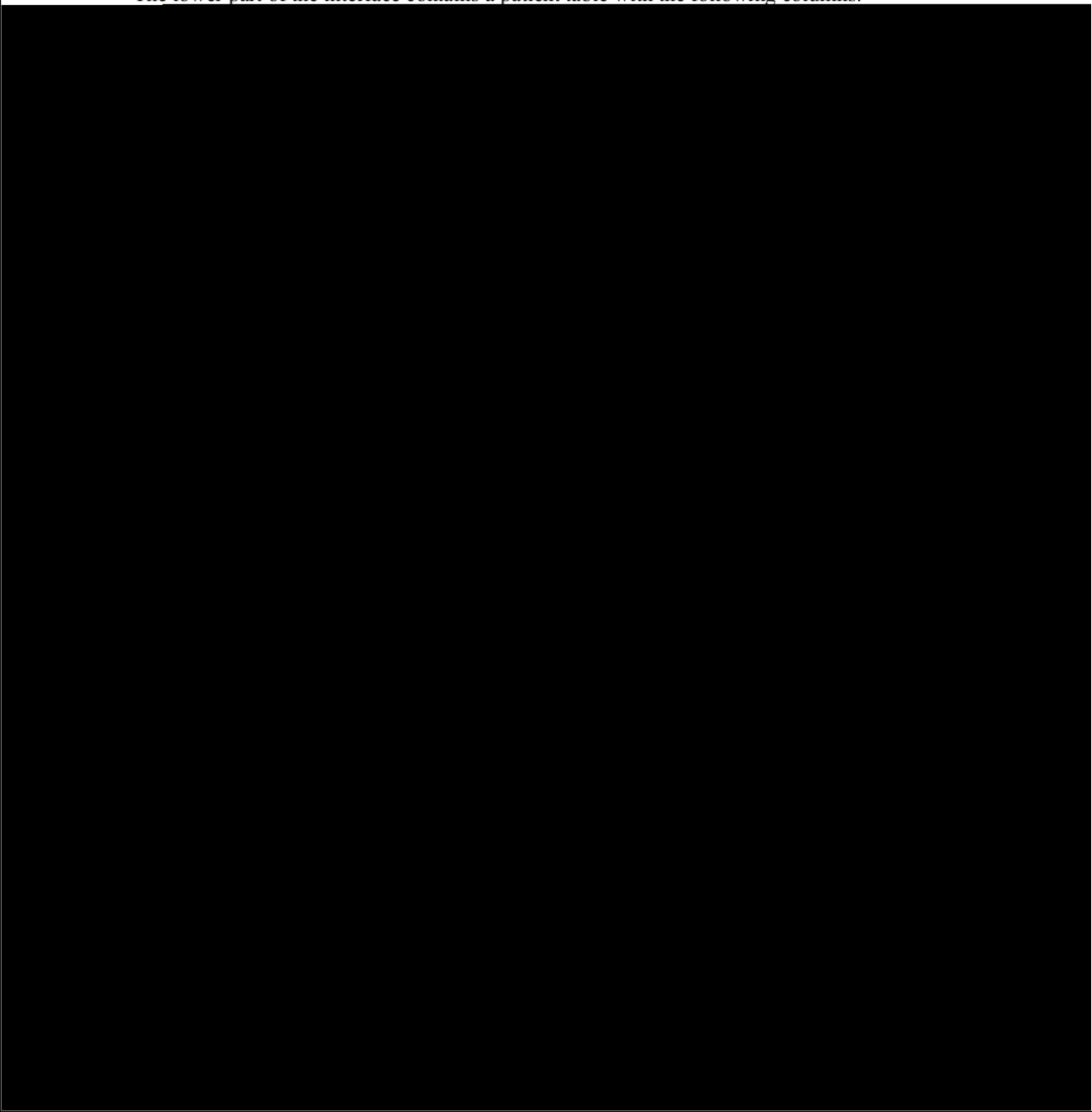
The **Name** field stores the patient's name, while the **Phone** field provides the contact information needed for communication.

The **Address** text area allows the patient's full address to be stored.

The **Date of Birth** field includes a date-selection control, helping users enter a properly formatted date instead of manually typing it.

A drop-down selection is provided for **Gender**, while the **Allergies** section provides space to record relevant allergy information.

The lower part of the interface contains a patient table with the following columns:



3.3.5 Appointment Management Interface

The screenshot shows the 'Appointments' management interface. On the left is a blue sidebar with the 'SUNRISE DENTAL' logo and navigation links: Patients, Treatment, Prescription, Dashboard, and a Logout button. The main content area has a title 'Appointments' and a red close button. It features four input fields: 'Date' with a calendar icon, 'Patient' with a dropdown arrow, 'Time' with a calendar icon, and 'Treatments' with a dropdown arrow. Below these fields are three buttons: 'Save', 'Edit', and 'Delete'. Underneath the buttons is a section titled 'Treatments List' containing a table with the following columns: ID, Patient Name, Treatments, Date, and Time. The table body is currently empty.

Figure 9-Appointment

The Appointment Management interface handles the scheduling of dental appointments.

The current screen contains fields for:

- **Date**
- **Patient**
- **Time**
- **Treatment**

The Patient and Treatment fields are provided using drop-down lists. This is useful because users can select information already available in the system rather than repeatedly typing the same patient or treatment names.

A date-selection control is also provided for appointment scheduling.

The interface includes three main operations:

3.3.5.1 Save

The **Save** button is intended to store a newly entered appointment.

3.3.5.2 *Edit*

The **Edit** button allows an existing appointment to be modified where details such as the date, time or treatment have changed.

3.3.5.3 *Delete*

The **Delete** button allows an unwanted or cancelled appointment record to be removed.

The lower part of the interface contains an appointment list with columns for:

- ID
- Patient Name
- Treatments
- Date
- Time

This provides users with a clear overview of scheduled appointments.

The interface is directly relevant to one of the main problems identified in the clinic scenario: appointments are currently managed manually, which contributes to double bookings and inefficient record handling.

3.3.7 Treatment Management Interface

Treatments

Name Treatment Medicines

Treatments List

ID	Patient Name	Treatments	Cost	Medicines	Quantity
----	--------------	------------	------	-----------	----------

Figure 10-Treatments

The Treatment Management interface is intended to maintain information relating to treatments provided by Sunrise Dental Clinic.

The interface currently contains:

- **Name**
- **Treatment**
- **Medicines**
- **Save**
- **Edit**
- **Delete**

The use of Save, Edit and Delete operations allows treatment-related records to be created and maintained from a single interface.

A table is provided below the input section. The visible columns include:

- ID
- Patient Name
- Treatments
- Cost
- Medicines
- Quantity

This makes treatment-related information available in a structured tabular format.

The Treatment interface is particularly important because the assessment requires the system to calculate bills based on the treatment type and consultation fee.

For the final implementation, treatment information should ideally include a clearly defined treatment price.

For example:

Treatment	Treatment Cost
Dental Cleaning	LKR 5,000
Filling	LKR 8,000
Extraction	LKR 10,000

The billing section could then automatically retrieve the stored treatment price rather than requiring users to manually type the treatment fee each time.

3.3.9 Prescription Management Interface

SUNRISE DENTAL

Patients
Treatment
Appointment
Dashboard
Logout

Prescriptions

Treatment

Patient

Medicines

Cost

Quantity

Save
Delete
Edit

Treatments List

ID	Patient Name	Treatments	Cost	Medicines	Quantity
----	--------------	------------	------	-----------	----------

Figure 11-Prescriptions

The Prescription interface is an additional feature of the system and extends the functionality beyond simple appointment management.

The current interface contains:

- Treatment
- Patient
- Medicines
- Cost
- Quantity

Three main controls are provided:

- Save
- Delete
- Edit

scription to be associated with

The Patient field uses a drop-down selection, allowing the pres
an existing patient.

entered, while Quantity

The Medicines section allows the medicine information to be e
identifies the amount prescribed.

g to the prescription.

The Cost field provides additional financial information relat

The lower table displays:

- ID
- Patient Name
- Treatments
- Cost
- Medicines
- Quantity

This makes it possible for clinic staff to view stored prescription information in a single organized area.

Prescription management is not one of the minimum functions specifically required in the scenario, but it adds useful functionality to the dental management system by allowing treatment-related medicine information to be maintained together with patient information.

3.3.10 Navigation and Logout

A consistent side navigation menu is used across the major screens.

Depending on the selected page, users can navigate to:

- Patients
- Treatment
- Prescription
- Appointment
- Dashboard

A **Logout** option appears at the bottom of the menu.

The consistent positioning of the navigation options is useful because users do not need to learn a different layout for every screen.

Logout also provides a clear way to end the current user's access to the system.

3.5 Input Validation

Validation should be applied to the main interfaces before data is stored.

Examples relevant to the current application include:

Input	Recommended Validation
Username	Must not be empty
Password	Must not be empty
Patient Name	Required
Phone Number	Must contain a valid number format
Address	Required
DOB	Must be a valid past date
Gender	A valid option must be selected
Appointment Date	Required valid date
Appointment Time	Required
Patient	Must be selected
Treatment	Must be selected
Appointment Number	Must be unique
Cost	Numeric and non-negative
Quantity	Positive integer
Dentist	Must be selected

Where invalid information is entered, the system should display understandable messages such as:

“Please enter the patient name.”

“Please select a treatment.”

“Invalid phone number.”

“Appointment number already exists.”

“Selected dentist is unavailable at this time.”

These messages would prevent users from saving incomplete or invalid information. **Evidence to add to the final report:** screenshots showing at least two or three validation messages.

3.6 Database Integration

The assessment requires the application to use a **proper database** for storing system information.

Based on the interfaces already created, the database would logically need to store information relating to:

- Users
- Patients
- Appointments
- Treatments
- Prescriptions
- Billing

A suitable relational structure may include:

Patient Table

- patient_id
- patient_name
- phone
- address
- dob
- gender
- allergies

Appointment Table

- appointment_id³
- appointment_number
- patient_id
- dentist_id
- treatment_id
- appointment_date
- appointment_time

Treatment Table

- ³ treatment_id
- treatment_name
- treatment_cost

Prescription Table

- prescription_id
- patient_id
- treatment_id
- medicines
- quantity
- cost

User Table

- user_id
- username
- password
- ⁷role

Primary keys can uniquely identify records, while foreign keys can connect appointments, treatments, patients and prescriptions.

The current screenshots show tables intended to display stored records, but screenshots alone do not confirm the database technology or database schema used. Therefore, the final report should include a database screenshot or ER diagram showing the actual implemented database.

4 Testing

The main system performs its main

om being entered. This is particularly

tails, appointments, treatments and

test rationale, test plan, test data,

test plan, test data, test data,

of functional testing, validation testing and

ks whether features such as login, patient

s expected. Validation testing checks whether

to confirm that the system responds

responds appropriately to errors.

Testing was carried out to ensure that the system

functions correctly and prevents invalid information from

important because the system manages patient data

prescriptions. The assessment specifically requires a

application to be tested. Test cases should be developed

4.1 Test Strategy and Rationale

The system should be tested using a combination

automated unit testing. Functional testing checks

registration and appointment management work as

incorrect or incomplete information is rejected.

Both valid and invalid test data are used. Valid data

accepted, while invalid data confirms that the system

The main areas tested are:

- User login
- Patient management
- Appointment management
- Treatment management
- Prescription management
- Billing calculations

• Logout and navigation

original clinic problem includes

Testing is especially important for appointments because the

double bookings, lost records and billing errors.

4

4.2 Test-Driven Development

Test-Driven Development follows the cycle:

Red → Green → Refactor

Red: A test is created for the required behaviour and initially fails

Green: The required program code is developed until the test passes.

Refactor: The code is improved without changing the successful result.

For example, before accepting an appointment, a test can be created to confirm that a patient and treatment must be selected. If the appointment is incomplete, the system should reject it. Once validation is added and the test passes, the code can be improved while preserving the same behaviour.

TDD can also be applied to bill calculation. If the treatment fee is **LKR 8,000** and the consultation fee is **LKR 2,000**, the expected total is:

$$8,000 + 2,000 = 10,000$$

The automated test should therefore verify that the system returns **LKR 10,000**.

4.3 Test Plan

Test ID	Module	Test Data	Expected Result
TC01	Login	Valid username/password	Dashboard displayed
TC02	Login	Invalid password	Error message displayed
TC03	Login	Empty fields	Login prevented
TC04	Patient	Valid patient details	Patient saved
TC05	Patient	Empty patient name	Validation message
TC06	Patient	Invalid phone number	Validation message
TC07	Appointment	Valid appointment data	Appointment saved
TC08	Appointment	Missing patient	Save prevented
TC09	Appointment	Duplicate appointment no.	Error displayed
TC10	Appointment	Same dentist/date/time	Double booking prevented
TC11	Appointment	Edit existing appointment	Record updated
TC12	Appointment	Delete appointment	Record removed
TC13	Treatment	Valid treatment	Treatment saved
TC14	Prescription	Valid prescription	Prescription saved
TC15	Prescription	Negative quantity	Validation error
TC16	Billing	8000 + 2000	Total = 10000
TC17	Search	Invalid appointment no.	Not-found message
TC18	Logout	Click Logout	Login screen displayed

4.4 Test Data and Application

Example valid patient data:

- Name: Nimal Perera
- Phone: 0771234567
- Address: Colombo
- DOB: 15/06/1995
- Gender: Male
- Allergies: None

Example invalid data:

- Name: blank
- Phone: ABC123
- Appointment patient: not selected
- Quantity: -5

When valid information is entered, the relevant record should be stored and displayed in the table. Invalid information should produce a clear validation message and should not be saved.

For the final report, the **Actual Result** and **Pass/Fail** status should only be completed after each test is actually executed.

4.5 Test Automation

Automated unit testing can be carried out using **JUnit**. Suitable automated tests include:

- login validation;
- patient validation;
- appointment validation;
- duplicate appointment detection;
- bill calculation.

A simple billing test can verify that:

```
@Test
void testBillCalculation() {
    double treatmentFee = 8000.00;
    double consultationFee = 2000.00;
    assertEquals(10000.00, treatmentFee + consultationFee);
}
```

Running automated tests after changes helps identify problems quickly and reduces repeated manual testing.

4.6 Testing Evaluation

The testing approach checks both normal and incorrect user behavior. Manual functional testing is useful for checking the graphical interfaces. While automated tests are suitable for business rules such as calculations and validation.

The final report should include screenshots showing validation messages and successful JUnit test execution. These results will provide evidence that the system has been properly tested rather than only describing the intended test process.

1 JUnit test classes make it super easy to run tests automatically after you tweak your code, so you can check everything's still working right away. continuous integration workflows automatically run tests whenever someone commits to the repo, keeping the codebase stable and bug-free snapshots of JUnit test outcomes show all tests passing, which means the business logic is on point and the system's stable.

this organized testing method makes sure we check all the important stuff that the modules need to do, and we're on track with what they're supposed to do using automated tests made the dev work smoother and kept the code solid and the system dependable.

5 Version Control and Documentation

The project was developed in **NetBeans IDE** and its built-in **Git Management System**. The project was

initialize the local repository, stage files, create commits and push changes to GitHub.

repository is: The current public repository is:

named **A77A5H/Sunrise-Dental-Clinic-Management-System-git** <https://github.com/A77A5H/Sunrise-Dental-Clinic-Management-System-git>

It is noted that the original repository was lost, including the earlier commit history. An important limitation

is that the original repository was lost, including the earlier commit history. An important limitation

the current repository was created as a replacement using the latest available project files. The dates and commits visible in this repository represent the period after version control was re-established and should not be considered the complete development history. No attempt was made to recreate or backdate the lost commits.

The Git workflow used in NetBeans can be summarized as:

Modify Project → Stage Files → Commit Changes → Push to GitHub

The repository was connected to GitHub through the NetBeans **Git > Remote > Push** function.

Authentication was completed using a GitHub Personal Access Token. Meaningful commit messages were used to identify later modifications and preserve updated versions of the project.

Version control provides several advantages, including source-code backup, change tracking,

identification of different software versions and easier recovery from development errors. The

assessment requires evidence of repository creation, several software versions, version-control techniques and workflow documentation.

The final report should include screenshots of the **GitHub repository, commit history and NetBeans Git interface**. If branches, pull requests or GitHub Actions are subsequently used, screenshots of these should also be included as additional evidence of the version-control workflow.

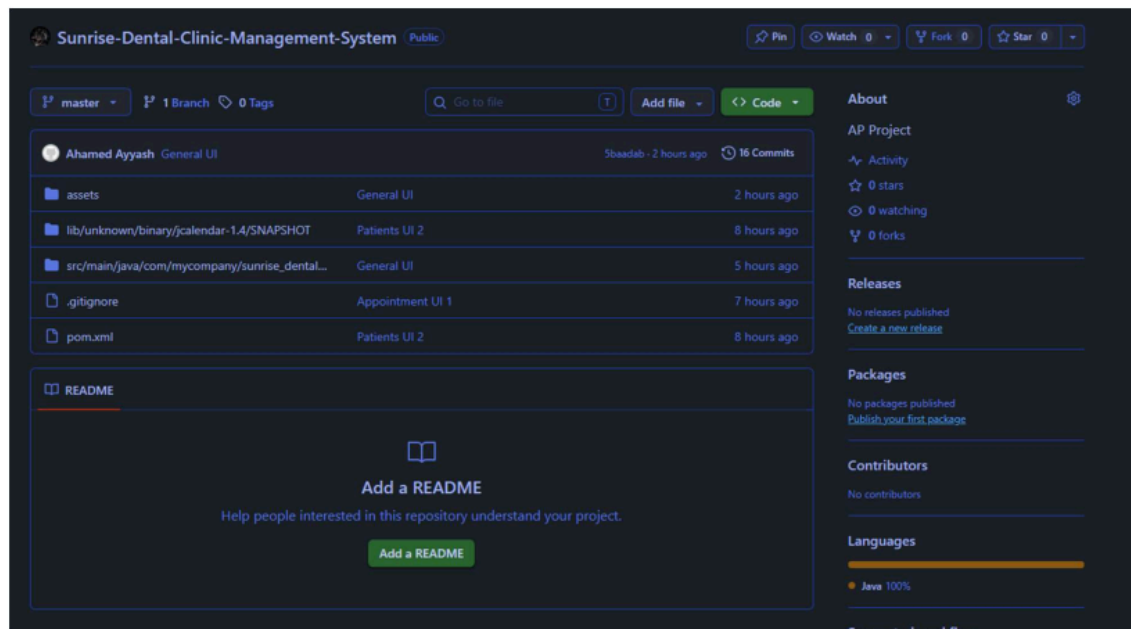
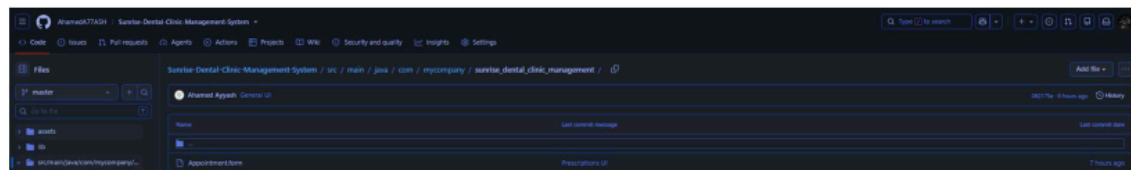


Figure 12-Public GitHub Repository I



6 Conclusion

6.1 Project Outcomes

The Sunrise Dental Clinic Management System was developed to make the clinic's daily work easier and more organized. The system includes login, patient management, appointments, treatments, prescriptions and a dashboard. It helps reduce the problems that can happen with manual records. The interface is also simple and easy to use. Overall, the system provides a useful base for managing the clinic in a more efficient way.

6.2 Challenges and Solutions

There were a few challenges during the development process. One of the main difficulties was keeping the different parts of the system connected and consistent. It was also important to make sure the interfaces were clear and easy to use. Another challenge was losing the original GitHub repository and its earlier commit history. A new repository was created using the latest project files so version control could continue. These issues helped improve the way the project was organized and managed.

6.3 Critical Reflection

This project gave me a better understanding of how Java applications are planned, designed and tested. UML diagrams helped me understand the structure of the system before development. Separating the system into different modules also made the project easier to manage. Testing showed the importance of checking both valid and invalid inputs. Git and GitHub were also useful for tracking changes and keeping the project backed up.

6.4 Future Improvements

The system can be improved further in the future. Possible improvements include:

- Adding a complete billing and receipt system
- Improving appointment search and dentist scheduling
- Adding stronger input validation
- Developing more detailed reports
- Adding web services and distributed features
- Improving database security
- Adding email or SMS appointment reminders
- Creating a mobile or web-based version of the system

References

- Bouzidi, A., Haddar, N., Ben-Abdallah, M. and Haddar, K. (2020). Toward the Alignment and Traceability between Business Process and Software Models. Proceedings of the 22nd International Conference on Enterprise Information Systems, pp.701–708. doi:<https://doi.org/10.5220/0009004607010708>.
- Imane Essebaa and Salima Chantit (2018). Tool Support to Automate Transformations from SBVR to UML Use Case Diagram. pp.525–532. doi:<https://doi.org/10.5220/0006817705250532>.
- Jick Yeung (2017). Use Case Diagram notations guide - Visual Paradigm. [online] Visual Paradigm Community Circle. Available at: <https://circle.visual-paradigm.com/docs/uml-and-sysml/use-case-diagram/use-case-diagram-notations-guide/>.
- Creately (2022). UML Class Diagram Relationships Explained with Examples | Creately. [online] creately.com. Available at: <https://creately.com/guides/class-diagram-relationships/>.
- Visual Paradigm (2019). UML Association vs Aggregation vs Composition. [online] [Visual-paradigm.com](https://www.visual-paradigm.com). Available at: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-aggregation-vs-composition/>.
- Muhamad, Z.H., Abdulmonim, D.A. and Alathari, B. (2019). An integration of uml use case diagram and activity diagram with Z language for formalization of library management system. International Journal of Electrical and Computer Engineering (IJECE), 9(4), p.3069. doi:<https://doi.org/10.11591/ijece.v9i4.pp3069-3076>.
- Singh, M., Sharma, A.K. and Saxena, R. (2016). Formal Transformation of UML Diagram: Use Case, Class, Sequence Diagram with Z Notation for Representing the Static and Dynamic Perspectives of System, *Advances in Intelligent Systems and Computing*, pp.25–38. doi:https://doi.org/10.1007/978-981-10-0135-2_3.
- tutorialspoint.com (2019). UML - Basic Notations. [online] www.tutorialspoint.com. Available at: https://www.tutorialspoint.com/uml/uml_basic_notations.htm.
- Kirill Fakhroutdinov (2014). Use case diagrams are UML diagrams describing units of useful functionality (use cases) performed by a system in collaboration with external users (actors). [online] [Uml-diagrams.org](http://uml-diagrams.org). Available at: <https://www.uml-diagrams.org/use-case-diagrams.html>.

- Anon, (n.d.). UML 2 Use Case Diagramming Guidelines – The Agile Modeling (AM) Method. [online] Available at: <https://agilemodeling.com/style/usecasediagram.htm>.
- Hamza, Z.A. and Hammad, M. (2020). Generating Test Sequences from UML Use Case Diagram: A Case Study. [online] IEEE Xplore. doi:<https://doi.org/10.1109/IEEECONF51154.2020.9319979>.
- www.uml-diagrams.org. (n.d.). UML composite aggregation (composition) is relationship between a property and at most one composite object with responsibility for the existence and storage of that property (part). [online] Available at: <https://www.uml-diagrams.org/composition.html>.
- Sparxsystems.com. (2025). Sequence Element Activations | Enterprise Architect User Guide. [online] Available at: https://sparxsystems.com/enterprise_architect_user_guide/17.1/modeling_languages/sequenceelementactivation.html [Accessed 21 Aug. 2025].
- Sequence Diagrams. (n.d.). Available at: <https://www.se.rit.edu/~swen-261/slides/Sequence%20Diagrams.pdf> [Accessed 21 Aug. 2025].
- sparxsystems.com. (n.d.). Sequence Diagram - UML 2 Tutorial | Sparx Systems. [online] Available at: <https://sparxsystems.com/resources/tutorials/uml2/sequence-diagram.html>.
- EAI Endorsed Trans e-Learn (2023) ‘Design of UML Diagrams for WEBMED - Healthcare Service System Services’, vol. 8, no. 1, p. e5. Available at: <https://publications.eai.eu/index.php/el/article/view/3015> (Accessed: 21 August 2025).
- Rodina, D. (2020) ‘Billing System (UML Component Diagram)’, Softwareideas.net. Available at: <https://www.softwareideas.net/a/1590/billing-system-uml-component-diagram-> (Accessed: 21 August 2025).
- Stack Overflow (n.d.) ‘Design Patterns Web Based Applications’. Available at: <https://stackoverflow.com/questions/3541077/design-patterns-web-based-applications> (Accessed: 21 August 2025).
- Java Design Patterns (n.d.) ‘Model-View-Controller’. Available at: <https://java-design-patterns.com/patterns/model-view-controller/> (Accessed: 21 August 2025).
- Baeldung (n.d.) ‘MVC Servlet JSP’. Available at: <https://www.baeldung.com/mvc-servlet-jsp> (Accessed: 21 August 2025).
- Refactoring Guru (n.d.) ‘Design Patterns in Java’. Available at: <https://refactoring.guru/design-patterns/java> (Accessed: 21 August 2025).

- DigitalOcean (n.d.) 'Java Design Patterns Example Tutorial'. Available at: <https://www.digitalocean.com/community/tutorials/java-design-patterns-example-tutorial> (Accessed: 21 August 2025).
- DigitalOcean (n.d.) 'RESTful Web Services Tutorial Java'. Available at: <https://www.digitalocean.com/community/tutorials/restful-web-services-tutorial-java> (Accessed: 21 August 2025).

ava (n.d.) 'Java RESTful Web Services Tutorial for Beginner with Jersey and Tomcat'. Available at: https://www.codejava.net/java-ee/web-services/java-web-services-tutorial-for-beginner-with-jersey-and-tomcat (Accessed: 21 August 2025).	• CodeJava
restful-	Tomcat'. Available at: https://www.codejava.net/java-ee/web-services/java-web-services-tutorial-for-beginner-with-jersey-and-tomcat (Accessed: 21 August 2025).
August	2025).
at:	• Jakarta EE (n.d.) 'RESTful Web Services'. Available at: https://jakarta.ee/learn/docs/jakartaee-tutorial/current/websvcs/rest/rest.html (Accessed: 21 August 2025).
ble at:	• Creately (n.d.) 'Billing System UML Diagram Example'. Available at: https://creately.com/diagram/example/i9je5vs82/billing-system (Accessed: 21 August 2025).
August	2025).
at:	• JetBrains (n.d.) 'TDD with IntelliJ IDEA'. Available at: https://www.jetbrains.com/help/idea/tdd-with-intellij-idea.html (Accessed: 21 August 2025).
August	2025).
Mock,	• Makariev (2023) 'Test-Driven Development Spring Boot Guide to Unit, Integration Tests'. Available at: https://www.makariev.com/blog/test-development-spring-boot-guide-to-unit-mock-integration-tests/ (Accessed: 21 August 2025).
-driven-	2025).
August	2025).
le at:	• YouTube (2021) 'Test Driven Development Tutorial'. Available at: https://www.youtube.com/watch?v=eMU_hninZAs (Accessed: 21 August 2025).
5).	2025).
at:	• Java Code Geeks (n.d.) 'JUnit Tutorials'. Available at: https://www.javacodegeeks.com/junit-tutorials (Accessed: 21 August 2025).
lable at:	• DEV.to (2020) 'How to Use JUnit and Mockito for Unit Testing in Java'. Available at: https://dev.to/whathebea/how-to-use-junit-and-mockito-for-unit-testing-in-java (Accessed: 21 August 2025).
-4pjb	2025).
at:	• Vogella (n.d.) 'JUnit Tutorial'. Available at: https://www.vogella.com/tutorials/JUnit/article.html (Accessed: 21 August 2025).
25).	2025).

- GitHub Docs (n.d.) 'Building and Testing Java with Gradle'. Available at: <https://docs.github.com/actions/automating-builds-and-tests/building-and-testing-java-with-gradle> (Accessed: 21 August 2025).
- GitHub Docs (n.d.) 'Building and Testing Java with Maven'. Available at: <https://docs.github.com/en/actions/how-tos/use-cases-and-examples/building-and-testing/building-and-testing-java-with-maven> (Accessed: 21 August 2025).
- DEV.to (2021) 'Java App CI/CD Using GitHub Actions'. Available at: <https://dev.to/robudexit/java-app-cicd-using-github-actions-12n1> (Accessed: 21 August 2025).
- Vertabelo Blog (n.d.) 'Billing System Database Model'. Available at: <https://vertabelo.com/blog/billing-system-database-model/> (Accessed: 21 August 2025).
- Student Project Guide (n.d.) 'Electricity Billing System Database Design'. Available at: <https://studentprojectguide.com/project-report/database-design/electricity-billing-system-database-design/> (Accessed: 21 August 2025).
- 99X Engineering (2021) 'An Introduction to MVC Architecture'. Available at: <https://engineering.99x.io/an-introduction-to-mvc-architecture-22ff684d48c8> (Accessed: 21 August 2025).
- University College Dublin (n.d.) 'Harvard Style Guide'. Available at: <https://libguides.ucd.ie/harvardstyle> (Accessed: 21 August 2025).
- Universitas Gadjah Mada (n.d.) 'Harvard Referencing Style Guide'. Available at: <https://lib.ugm.ac.id/en/harvard-referencing-style/> (Accessed: 21 August 2025).
- Granthaalayah Publication (n.d.) 'Article on Arts Journal'. Available at: <https://www.granthaalayahpublication.org/Arts-Journal/ShodhKosh/article/view/3571> (Accessed: 21 August 2025).
- British Journal of Healthcare Assistants (2019) 'Article Title', vol. 13, no. 9, pp. 444. ... Available at: <http://www.magonlinelibrary.com/doi/10.12968/bjha.2019.13.9.444> (Accessed: 21 August 2025).
- SiELE Journal (n.d.) 'Article Title'. Available at: <https://jurnal.usk.ac.id/SiELE/article/view/34776> (Accessed: 21 August 2025).
- CiteThisForMe (n.d.) 'Harvard Citation Generator'. Available at: <https://www.citethisforme.com/citation-generator/harvard> (Accessed: 21 August 2025).

- Norwegian University of Science and Technology (n.d.) 'Academic Writing Harvard Style'. Available at: <https://i.ntnu.no/en/academic-writing/harvard> (Accessed: 21 August 2025).
- Tutorialspoint (n.d.) 'MVC Pattern'. Available at: https://www.tutorialspoint.com/design_pattern/mvc_pattern.htm (Accessed: 21 August 2025).
- Grammarly (n.d.) 'Harvard Style Citations'. Available at: <https://www.grammarly.com/blog/citations/harvard-style/> (Accessed: 21 August 2025).
- EasyBib (n.d.) 'Harvard Referencing Guide'. Available at: <https://www.easybib.com/guides/citation-guides/harvard-referencing/> (Accessed: 21 August 2025).
- Scribbr (n.d.) 'Harvard Referencing Style'. Available at: <https://www.scribbr.co.uk/referencing/harvard-style/> (Accessed: 21 August 2025).
- Taylor & Francis (2022) 'Chapter 24', In: Book Title. Available at: <https://www.taylorfrancis.com/books/9781003287124/chapters/10.1201/9781003287124-24> (Accessed: 21 August 2025).
- Taylor & Francis (2022) 'Chapter 19', In: Book Title. Available at: <https://www.taylorfrancis.com/books/9781003287124/chapters/10.1201/9781003287124-19> (Accessed: 21 August 2025).
- Taylor & Francis (2022) 'Chapter 36', In: Book Title. Available at: <https://www.taylorfrancis.com/books/9781003287124/chapters/10.1201/9781003287124-36> (Accessed: 21 August 2025).
- Springer (2023) 'Article Title', Springer. Available at: <https://link.springer.com/10.1007/s10515-023-00385-x> (Accessed: 21 August 2025).
- IEEE Xplore (2023) 'Article Title', IEEE Xplore. Available at: <https://ieeexplore.ieee.org/document/10350718/> (Accessed: 21 August 2025).

ORIGINALITY REPORT

14%

SIMILARITY INDEX

2%

INTERNET SOURCES

0%

PUBLICATIONS

13%

STUDENT PAPERS

PRIMARY SOURCES

1**Submitted to University of Wales Institute, Cardiff**

Student Paper

11%**2****github.com**

Internet Source

1%**3****Submitted to Kaplan Professional**

Student Paper

<1%**4****Submitted to Malta College of Arts, Science and Technology**

Student Paper

<1%**5****Submitted to Edith Cowan University**

Student Paper

<1%**6****Submitted to The Robert Gordon University**

Student Paper

<1%**7****www.clicdata.com**

Internet Source

<1%**8****Submitted to Asia Pacific International College**

Student Paper

<1%**9****www.vijaythapa.com.np**

Internet Source

<1%

Exclude quotes

On

Exclude matches

Off

Exclude bibliography

On